KernelVerse

OS Concepts Simulator — Python Engine Testing Guide

This guide explains how to test every Python simulation engine in KernelVerse: CPU Scheduling, Memory Management, Paging, Deadlocks, and Disk Scheduling. It covers running the FastAPI backend, sending requests with **curl**, interpreting responses, and running engines directly in Python.

Item	Detail
Backend	FastAPI + Pydantic (Python 3.13+)
Engine directory	api/app/engines/
API entry point	api/app/main.py — POST /simulate
Local server	http://127.0.0.1:8000
Engines covered	CPU scheduling, memory management, paging, deadlocks, disk scheduling

1. Prerequisites & Starting the Server

All engines live under **api/app/engines/** and are exposed through a single FastAPI endpoint: **POST /simulate** with a JSON body {"topic": "…", "payload": {…}}. The backend validates the input, runs the simulation, and returns a full step-by-step trace plus metrics.

1.1 Install dependencies

```
$ cd api
$ python -m venv .venv
$ source .venv/bin/activate        # Windows: .venv\Scripts\activate
$ pip install -r requirements.txt  # fastapi, uvicorn[standard], pydantic
```

1.2 Start the server

```
$ cd api
$ source .venv/bin/activate
$ uvicorn app.main:app --host 127.0.0.1 --port 8000 --reload
INFO:     Uvicorn running on http://127.0.0.1:8000
```

1.3 Quick health check

```
$ curl -s http://127.0.0.1:8000/health
{"status":"ok"}
```

If you see {"status": "ok"}, the backend is ready.

1.4 Common request shape

```
POST /simulate
Content-Type: application/json

{
  "topic": "cpu-scheduling",   # one of: cpu-scheduling, memory-management, paging, deadlocks, disk-scheduli
  "payload": { ... }          # engine-specific inputs
}
```

Every engine also exposes a validate(payload) function that returns a list of human-readable errors (empty list = valid). The API returns HTTP 422 with the errors if validation fails.

2. CPU Scheduling — *cpu_scheduling.py*

Simulates process execution on a single CPU, tick by tick. Supports **FCFS**, **SJF**, **SRTF**, **Priority**, **Round Robin (RR)**, and **Multilevel Queue (MLQ)**, with optional I/O blocking.

Endpoint: topic = "cpu-scheduling"
File: api/app/engines/cpu_scheduling.py

2.1 Test: FCFS (non-preemptive)

```
{
  "algorithm": "fcfs",
  "processes": [
    {"id": "P1", "arrival": 0, "burst": 5},
    {"id": "P2", "arrival": 1, "burst": 3},
    {"id": "P3", "arrival": 2, "burst": 1}
  ]
}
```

```
$ curl -s -X POST http://127.0.0.1:8000/simulate \
  -H "Content-Type: application/json" \
  -d '{"algorithm":"fcfs","processes":[{"id":"P1","arrival":0,"burst":5},{"id":"P2","arrival":1,"burst":3},{
```

Expected metrics: P1 runs 0–5, P2 runs 5–8, P3 runs 8–9. avg_waiting should be 2.0, avg_turnaround = 6.0.

2.2 Test: Round Robin (quantum = 2)

```
{
  "algorithm": "rr",
  "quantum": 2,
  "processes": [
    {"id": "P1", "arrival": 0, "burst": 5},
    {"id": "P2", "arrival": 1, "burst": 3},
    {"id": "P3", "arrival": 2, "burst": 1}
  ]
}
```

What to check: each process runs at most 2 ticks before going to the back of the ready queue. Look for context switches and quantum-expiry notes in the step trace.

2.3 Test: SRTF (preemptive shortest remaining time first)

```
{
  "algorithm": "srtf",
  "processes": [
    {"id": "P1", "arrival": 0, "burst": 6},
    {"id": "P2", "arrival": 2, "burst": 2}
  ]
}
```

What to check: when P2 arrives at tick 2 with remaining 2 < P1's remaining 4, P1 should be preempted and P2 should run next.

2.4 Test: MLQ (multilevel queue)

```
{
  "algorithm": "mlq",
  "quantum": 2,
  "processes": [
    {"id": "H1", "arrival": 0, "burst": 4, "priority": 1, "queue": 0},
    {"id": "L1", "arrival": 0, "burst": 4, "priority": 1, "queue": 1}
  ]
}
```

What to check: high-queue (queue 0) processes are always preferred over low-queue (queue 1). A low-queue process running when a high-queue process arrives gets preempted.

2.5 Test: with I/O blocking

```
{
```

```
  "algorithm": "rr",
  "quantum": 2,
  "processes": [
    {"id": "P1", "arrival": 0, "burst": 6,
     "io": {"after_cpu": 2, "duration": 3}},
    {"id": "P2", "arrival": 0, "burst": 4}
  ]
}
```

What to check: after P1 consumes 2 CPU ticks, it blocks for 3 ticks (I/O), then re-enters the ready queue. P2 should run while P1 is blocked.

2.6 Metrics to verify

Metric	Meaning
avg_waiting	Average time a process spends in the ready queue
avg_turnaround	Average finish_time – arrival_time
avg_response	Average first_start – arrival_time
throughput	Completed processes / total time
cpu_utilization	Busy ticks / total ticks
context_switches	Number of times the running process changed

3. Memory Management — *memory_management.py*

Simulates placing processes into memory holes. Supports **contiguous** allocation (one block per process) and **segmentation** (multiple named segments per process), with **first-fit**, **best-fit**, and **worst-fit** placement strategies.

Endpoint: topic = "memory-management"
File: api/app/engines/memory_management.py

3.1 Test: contiguous, first-fit

```
{
  "mode": "contiguous",
  "strategy": "first-fit",
  "memory_size": 100,
  "processes": [
    {"id": "P1", "size": 30},
    {"id": "P2", "size": 40},
    {"id": "P3", "size": 20}
  ]
}
```

```
$ curl -s -X POST http://127.0.0.1:8000/simulate \
  -H "Content-Type: application/json" \
  -d '{"mode":"contiguous","strategy":"first-fit","memory_size":100,"processes":[{"id":"P1","size":30},{"id"
```

Expected: P1 at address 0, P2 at 30, P3 at 70. memory_utilization = 0.9, no failures.

3.2 Test: best-fit vs worst-fit (compare fragmentation)

```
{
  "mode": "contiguous",
  "strategy": "best-fit",
  "memory_size": 100,
  "processes": [
    {"id": "P1", "size": 10},
    {"id": "P2", "size": 50},
    {"id": "P3", "size": 30}
  ]
}
```

What to check: best-fit picks the smallest hole that fits (tends to leave tiny useless holes). Repeat with "strategy": "worst-fit" and compare external_fragmentation.

3.3 Test: segmentation

```
{
  "mode": "segmentation",
  "strategy": "first-fit",
  "memory_size": 200,
  "processes": [
    {"id": "P1", "segments": [
        {"name": "code", "size": 40},
        {"name": "stack", "size": 30}
    ]}
  ]
}
```

What to check: each segment is placed independently. The response includes a segment_table with base and limit addresses for every segment.

3.4 Test: pre-existing holes (fragmentation)

```
{
  "mode": "contiguous",
  "strategy": "first-fit",
  "memory_size": 100,
  "holes": [{"start": 0, "size": 20}, {"start": 40, "size": 30}, {"start": 80, "size": 20}],
  "processes": [
    {"id": "P1", "size": 25},
    {"id": "P2", "size": 15}
```

```
    ]
  }
```

What to check: P1 (25) fits in the 30-unit hole at 40; P2 (15) fits in the 20-unit hole at 0 or 80. Watch failed_allocations when no hole is large enough.

3.5 Metrics to verify

Metric	Meaning
memory_utilization	Allocated bytes / memory_size
total_free	Total free bytes remaining
largest_free_hole	Size of the biggest contiguous free block
external_fragmentation	1 – (largest_free / total_free); higher = more fragmented
failed_allocations	Processes/segments that could not be placed
segment_table	base/limit per segment (segmentation mode only)

4. Paging — *paging.py*

Simulates page replacement with a fixed number of frames over a reference string. Supports **FIFO**, **LRU**, and **Optimal** replacement, plus an optional **TLB** (translation lookaside buffer).

Endpoint: topic = "paging"
File: api/app/engines/paging.py

4.1 Test: FIFO replacement

```
{
  "algorithm": "fifo",
  "reference_string": [1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5],
  "frames": 3,
  "tlb_size": 0
}
```

```
$ curl -s -X POST http://127.0.0.1:8000/simulate \
  -H "Content-Type: application/json" \
  -d '{"algorithm":"fifo","reference_string":[1,2,3,4,1,2,5,1,2,3,4,5],"frames":3,"tlb_size":0}'
```

Expected: classic Belamy example — page_faults = 9, fault_rate = 0.75.

4.2 Test: LRU replacement

```
{
  "algorithm": "lru",
  "reference_string": [1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5],
  "frames": 3,
  "tlb_size": 0
}
```

What to check: LRU evicts the least-recently-used page; compare fault count with FIFO above.

4.3 Test: Optimal replacement

```
{
  "algorithm": "optimal",
  "reference_string": [1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5],
  "frames": 3,
  "tlb_size": 0
}
```

What to check: Optimal looks ahead in the reference string and evicts the page needed farthest in the future — this gives the lowest possible fault count.

4.4 Test: with TLB

```
{
  "algorithm": "fifo",
  "reference_string": [1, 2, 1, 2, 3, 1, 2],
  "frames": 3,
  "tlb_size": 4
}
```

What to check: repeated accesses to pages already in the TLB give tlb_hits > 0. A TLB hit avoids a page-table walk even before checking frames.

4.5 Metrics to verify

Metric	Meaning
page_faults	Number of times a page was not in memory
frame_hits	Number of times a page was already in a frame
fault_rate	page_faults / total_references
tlb_hits	TLB cache hits (translation cached)

Metric	Meaning
tlb_misses	TLB lookups that missed
total_references	Length of the reference string

5. Deadlocks — *deadlocks.py*

Two modes: **banker** (avoidance via the Banker's safety algorithm) and **detection** (wait-for-graph cycle detection).

Endpoint: topic = "deadlocks"
File: api/app/engines/deadlocks.py

5.1 Test: Banker's algorithm — SAFE state

```
{
  "mode": "banker",
  "processes": ["P0","P1","P2","P3","P4"],
  "resources": [{"id":"A","total":10},{"id":"B","total":5},{"id":"C","total":7}],
  "available": {"A":3,"B":3,"C":2},
  "allocation": {
    "P0": {"A":0,"B":1,"C":0},
    "P1": {"A":2,"B":0,"C":0},
    "P2": {"A":3,"B":0,"C":2},
    "P3": {"A":2,"B":1,"C":1},
    "P4": {"A":0,"B":0,"C":2}
  },
  "max": {
    "P0": {"A":7,"B":5,"C":3},
    "P1": {"A":3,"B":2,"C":2},
    "P2": {"A":9,"B":0,"C":2},
    "P3": {"A":2,"B":2,"C":2},
    "P4": {"A":4,"B":3,"C":3}
  }
}
```

```
$ curl -s -X POST http://127.0.0.1:8000/simulate \
  -H "Content-Type: application/json" \
  -d '{"mode":"banker","processes":["P0","P1","P2","P3","P4"],"resources":[{"id":"A","total":10},{"id":"B","
```

Expected: safe = true, deadlock = false, with a safe_sequence (e.g. P1 → P3 → P0 → P2 → P4). Every step shows Need ≤ Work and resources being released.

5.2 Test: Banker's algorithm — UNSAFE state

Use the same resources/processes but tighten available so no process can finish (e.g. "available": {"A":0,"B":0,"C":0} with needs > 0).

Expected: safe = false, deadlock = true, and unfinished_processes lists every process that cannot complete — a deadlock is possible.

5.3 Test: Detection — wait-for-graph cycle

```
{
  "mode": "detection",
  "processes": ["P0","P1","P2"],
  "resources": [{"id":"R1","total":1},{"id":"R2","total":1}],
  "available": {"R1":0,"R2":0},
  "allocation": {"P0":{"R1":1,"R2":0},"P1":{"R1":0,"R2":1},"P2":{"R1":0,"R2":0}},
  "max": {"P0":{"R1":1,"R2":1},"P1":{"R1":1,"R2":1},"P2":{"R1":0,"R2":0}}
}
```

What to check: P0 holds R1 and needs R2; P1 holds R2 and needs R1 → wait-for cycle P0 ↔ P1. deadlock should be true with deadlock_cycle showing the loop.

5.4 Test: Detection — no deadlock

Set "available": {"R1":1,"R2":1} so needs can be satisfied. **Expected:** deadlock = false, deadlock_cycle = [].

5.5 Metrics to verify

Metric	Meaning
safe	True if a safe sequence exists (banker mode)

Metric	Meaning
safe_sequence	Order in which processes can finish (banker mode)
deadlock	True if deadlock detected
deadlock_cycle	The cycle of processes (detection mode)
unfinished_processes	Processes that cannot finish (unsafe state)
algorithm	banker's safety algorithm / wait-for-graph cycle detection

6. Disk Scheduling — *disk_scheduling.py*

Simulates disk head movement across a set of requests. Supports **FCFS**, **SSTF**, **SCAN** (elevator), **C-SCAN** (circular), and **LOOK**.

Endpoint: topic = "disk-scheduling"
File: api/app/engines/disk_scheduling.py

6.1 Test: FCFS

```
{
  "algorithm": "fcfs",
  "head": 50,
  "disk_size": 200,
  "requests": [82, 170, 43, 140, 24, 16, 190],
  "direction": "right"
}
```

```
$ curl -s -X POST http://127.0.0.1:8000/simulate \
  -H "Content-Type: application/json" \
  -d '{"algorithm":"fcfs","head":50,"disk_size":200,"requests":[82,170,43,140,24,16,190],"direction":"right"
```

Expected: head visits 82 → 170 → 43 → 140 → 24 → 16 → 190 in that exact order; total_seek is the sum of absolute differences.

6.2 Test: SSTF

```
{
  "algorithm": "sstf",
  "head": 50,
  "disk_size": 200,
  "requests": [82, 170, 43, 140, 24, 16, 190],
  "direction": "right"
}
```

What to check: head always jumps to the closest remaining request. total_seek should be lower than FCFS for this dataset.

6.3 Test: SCAN (elevator)

```
{
  "algorithm": "scan",
  "head": 50,
  "disk_size": 200,
  "requests": [82, 170, 43, 140, 24, 16, 190],
  "direction": "right"
}
```

What to check: head moves right servicing 82, 140, 170, 190, reaches disk end (199), reverses, then services 43, 24, 16 moving left.

6.4 Test: C-SCAN (circular)

```
{
  "algorithm": "cscan",
  "head": 50,
  "disk_size": 200,
  "requests": [82, 170, 43, 140, 24, 16, 190],
  "direction": "right"
}
```

What to check: head moves right to the end, jumps back to 0, then services remaining requests from the start (uniform wait time, no starvation at one end).

6.5 Test: LOOK

```
{
  "algorithm": "look",
  "head": 50,
```

```
  "disk_size": 200,
  "requests": [82, 170, 43, 140, 24, 16, 190],
  "direction": "right"
}
```

What to check: like SCAN but the head only goes as far as the last request in each direction (no travel to the physical disk end).

6.6 Test: direction = left

```
{
  "algorithm": "scan",
  "head": 50,
  "disk_size": 200,
  "requests": [82, 170, 43, 140, 24, 16, 190],
  "direction": "left"
}
```

What to check: head first services requests below 50 (43, 24, 16), reaches disk start (0), reverses, then services 82, 140, 170, 190.

6.7 Metrics to verify

Metric	Meaning
total_seek	Sum of all head movements (seek distance)
average_seek	total_seek / number of requests
head_movements	Number of individual moves
sequence	Full path the head took, including the start position
seek_distances	Distance of each individual move

7. Direct Python Testing (no API server)

You can import and run any engine directly in Python without starting FastAPI. This is useful for unit testing, pytest, or quick experiments.

```
from engines import cpu_scheduling, memory_management, paging, deadlocks, disk_scheduling

# CPU scheduling
res = cpu_scheduling.run({
    "algorithm": "rr",
    "quantum": 2,
    "processes": [
        {"id": "P1", "arrival": 0, "burst": 5},
        {"id": "P2", "arrival": 1, "burst": 3},
    ],
})
print(res.metrics)
print([s.explanation for s in res.steps[:3]])

# Validation
errors = cpu_scheduling.validate({"algorithm": "fcfs", "processes": []})
print(errors)  # ['processes must be a non-empty list']
```

Run from the api/app/ directory so the engines package imports correctly. Each engine's run() returns a SimulationResult with .steps, .metrics, and .warnings.

8. Error Handling & Validation

If the payload is invalid, the API returns HTTP 422 with a list of error messages. Common cases:

- **Unknown algorithm/topic:** 422 listing valid values (e.g. fcfs, sjf, srtf, priority, rr, mlq).
- **Missing/empty processes:** 'processes must be a non-empty list'.
- **Duplicate process id:** "duplicate process id 'P1'".
- **Bad arrival/burst:** arrival must be ≥ 0; burst must be > 0.
- **RR without quantum:** 'quantum must be an integer ≥ 1'.
- **Memory size ≤ 0:** 'memory_size must be a positive integer'.
- **Disk head ≥ disk_size:** 'head must be less than disk_size'.

Example error response:

```
{
  "detail": [
    "processes[0].burst must be an integer > 0"
  ]
}
```

9. Quick Cheat Sheet — All Engines

Topic	topic value	Key payload fields	Algorithms / modes
CPU scheduling	cpu-scheduling	algorithm, processes[{id,arrival,burst,priority,queue,io}], quantum, tq	fcfs, sjf, srtf, priority, rr, mlq
Memory management	memory-management	mode, strategy, memory_size, processes[{id,size}|{id,segments}], holes	first-fit, best-fit, worst-fit; contiguous, segmentation
Paging	paging	algorithm, reference_string, frames, tlb_size	fifo, lru, optimal
Deadlocks	deadlocks	mode, processes, resources[{id,total}], available, allocation, request	banker, detection
Disk scheduling	disk-scheduling	algorithm, head, disk_size, requests, direction	fcfs, sstf, scan, cscan, look

---

End of testing guide. All engines live in api/app/engines/ and are served by api/app/main.py at **POST /simulate**.